LeetCode Java 终极完整版（数据结构+位运算+算法逻辑+Demo）

一、基础数据类型 & 包装类 & 常用数据结构（含方法）

分类	数据结构/类型	定义/声明	核心常用方法 / 操作	方法说明
基础数据类型	int	int a = 10;	+ - * / %; == != > < >= <=	加减乘除、取余；比较运算
	long	long a = 100L;	同 int	长整型，防止大数溢出
	double	double d = 3.14;	同 int	浮点型运算
	char	char c = 'a';	c == 'a'; c - '0'; c - 'a'	字符比较；字符转数字；转0~25下标
	boolean	boolean f = true;	&& || !	与、或、非运算
	byte/short	byte b = 1;	同 int	极少直接使用
包装类	Integer	Integer a = 10;	intValue(); valueOf(int i); parseInt(String s)	转int；int转Integer；字符串转int
	Character	Character c = 'A';	isLetter(c); isDigit(c); toUpperCase(c); toLowerCase(c)	判断字母；判断数字；转大小写
字符串	String	String s = "abc";	length(); charAt(i); substring(l, r); equals(t); toCharArray(); split(regex); contains(s)	长度；取第i位；截取子串；判断相等；转字符数组；分割；判断包含子串
	StringBuilder	StringBuilder sb = new StringBuilder();	append(x); insert(i, x); delete(l, r); deleteCharAt(i);	尾部追加；插入；删除区间；删除单个字符；替换区间；反

			replace(l, r, str); reverse(); charAt(i); setCharAt(i, ch); toString()	转；取字符；修改字符；转回String
	StringBuffer	StringBuffer sb = new StringBuffer();	同 StringBuilder	线程安全，刷题极少使用
数组	int[] / char[]	int[] a = new int[5];	a.length; a[i]; Arrays.sort(a); Arrays.binarySearch(a, k)	数组长度；访问/赋值；排序；二分查找
集合	List<Integer>	List<Integer> list = new ArrayList<>();	add(x); get(i); set(i, x); remove(i); size(); isEmpty(); contains(x)	尾部添加；取值；修改；删除；元素个数；判空；判断包含
	Set<Integer>	Set<Integer> set = new HashSet<>();	add(x); remove(x); contains(x); size();	添加（去重）；删除；判断存在；大小
	Map<K,V>	Map<Integer,Integer> map = new HashMap<>();	put(k,v); get(k); getOrDefault(k,d); containsKey(k); remove(k); size()	存键值对；取值；默认值；判断含key；删除key；键值对数量
栈/队列	Stack	Stack<Integer> st = new Stack<>();	push(x); pop(); peek(); isEmpty();	入栈；出栈；看栈顶；判空
	Queue	Queue<Integer> q = new LinkedList<>();	offer(x); poll(); peek(); isEmpty();	入队；出队；看队头；判空
	Deque	Deque<Integer> dq = new ArrayDeque<>();	offerFirst/Last; pollFirst/Last; peekFirst/Last	两端添加；两端删除；查看两端
	PriorityQueue	PriorityQueue<Integer> pq = new PriorityQueue<>();	offer(x); poll(); peek(); isEmpty();	入堆；取堆顶；看堆顶；判空
链表	ListNode	class ListNode {int val; ListNode next;}	node.val; node.next; while(node != null)	节点值；下一节点；遍历链表

二、位运算（算法必用）

运算类型	Java 符号	一句话逻辑	典型用途	核心操作Demo
与	a & b	两位都为1才是1，否则0	判断奇偶、消去最低位1	boolean isOdd(int x) { return (x & 1) == 1; }
或	a | b	只要有1就是1	设某些位为1	int setBit(int x) { return x | 1; } // 设最低位为1
异或	a ^ b	相同为0，不同为1	交换、找唯一奇数次数字	int swap(int a, int b) { a ^= b; b ^= a; a ^= b; return a; }
非	~a	0变1，1变0	极少直接使用	int reverseBit(int x) { return ~x; }
左移	a << 1	整体左移，低位补0	等价 ×2	int multiply2(int x) { return x << 1; }
右移	a >> 1	带符号右移，高位补符号位	等价 ÷2（向下取整）	int divide2(int x) { return x >> 1; }
高频技巧	-	常用位运算套路	快速处理二进制问题	int removeLast1(int x) { return x & (x - 1); } // 消去最低位1

三、高频算法（逻辑概述 + Java 核心Demo）

1. 双指针

核心逻辑：用两个下标从两头/同方向往中间走，减少一层循环，降低时间复杂度。

适用场景：回文判断、两数之和、接雨水、链表倒数节点。

```java
// Demo：回文字符串判断（极简可运行）
boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
```

```
            l++;
            r--;
        }
        return true;
    }
```

2. 滑动窗口

核心逻辑：维护一个左右边界的“窗口”，右边界扩张、左边界收缩，在窗口内计算目标结果，避免重复计算。

适用场景：最长无重复子串、最小覆盖子串、子数组和问题。

```
// Demo：最长无重复子串长度
int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, max = 0;
    for (int right = 0; right < s.length(); right++) {
        // 窗口内有重复字符，收缩左边界
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left++));
        }
        set.add(s.charAt(right));
        max = Math.max(max, right - left + 1);
    }
    return max;
}
```

3. 前缀和

核心逻辑：先预处理出“前i项和”数组，后续查询任意区间和时，直接用“后项前缀和 - 前项前缀和”，实现O(1)查询。

适用场景：子数组和为k、区间和快速查询。

```
// Demo：前缀和数组构建（后续可直接用）
int[] preSum(int[] nums) {
    int[] pre = new int[nums.length + 1]; // pre[0] = 0，方便计算
    for (int i = 0; i < nums.length; i++) {
        pre[i+1] = pre[i] + nums[i];
    }
    // 区间 [l, r] 和 = pre[r+1] - pre[l]
    return pre;
}
```

4. 二分查找

核心逻辑：针对有序数组，每次将查找范围砍掉一半，快速定位目标值或边界，时间复杂度O(logn)。

适用场景：有序数组搜索、旋转数组找最小值、求最值边界。

```
// Demo：有序数组查找目标值（找不到返回-1）
int binarySearch(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2; // 避免溢出，替代 (l+r)/2
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}
```

5. 递归

核心逻辑：自己调用自己，将大问题拆成若干个相同结构的小问题，必须有明确的终止条件（避免死循环）。

适用场景：树遍历、链表反转、斐波那契数列、回溯算法。

```
// Demo：二叉树中序遍历（递归版）
class TreeNode { int val; TreeNode left; TreeNode right; }
void inorder(TreeNode root) {
    if (root == null) return; // 终止条件
    inorder(root.left); // 左子树
    System.out.println(root.val); // 处理当前节点
    inorder(root.right); // 右子树
}
```

6. 回溯（暴力搜索）

核心逻辑：尝试一条路径→走不通就回退（撤销操作）→换另一条路径，穷举所有可能，找到符合条件的解。

适用场景：全排列、子集、组合、N皇后、分割回文串。

```java
// Demo：数组全排列（返回所有可能的排列）
public List<List<Integer>> permute(int[] nums) {
    List<List&lt;Integer&gt;&gt; res = new ArrayList<>();
    backtrack(nums, new boolean[nums.length], new ArrayList<>(), res);
    return res;
}
// 回溯核心方法：used标记已使用元素，path记录当前路径，res存结果
void backtrack(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> res) {
    if (path.size() == nums.length) { // 终止条件：路径长度等于数组长度
        res.add(new ArrayList<>(path)); // 保存结果
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue; // 跳过已使用元素
        used[i] = true; // 标记使用
        path.add(nums[i]); // 加入路径
        backtrack(nums, used, path, res); // 递归
        path.remove(path.size()-1); // 撤销操作（回退）
        used[i] = false; // 取消标记
    }
}
```

7. DFS（深度优先搜索）

核心逻辑：一条路走到黑，走到尽头再回头，遍历所有分支（多用递归或栈实现）。

适用场景：树遍历、图遍历、岛屿数量、路径搜索。

```java
// Demo：岛屿数量（DFS版，遍历并标记已访问岛屿）
void dfs(char[][] grid, int i, int j) {
    // 终止条件：越界或当前不是岛屿（'0'）
    if (i < 0 || j < 0 || i >= grid.length || j >= grid[0].length || grid[i][j] == '0')
        return;
    grid[i][j] = '0'; // 标记为已访问（避免重复遍历）
    // 上下左右四个方向递归
    dfs(grid, i+1, j);
    dfs(grid, i-1, j);
    dfs(grid, i, j+1);
    dfs(grid, i, j-1);
}
```

8. BFS（广度优先搜索）

核心逻辑：一层一层往外扩散，按“距离”从小到大遍历（多用队列实现），适合找最短路径。

适用场景：二叉树层序遍历、最短路径、扩散类问题。

```
// Demo：二叉树层序遍历（BFS版，按层输出）
void bfs(TreeNode root) {
    if (root == null) return;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root); // 根节点入队
    while (!q.isEmpty()) {
        TreeNode node = q.poll(); // 出队当前节点
        System.out.println(node.val); // 处理当前节点
        // 左右子节点入队（按层遍历）
        if (node.left != null) q.offer(node.left);
        if (node.right != null) q.offer(node.right);
    }
}
```

9. 动态规划（DP）

核心逻辑：记录子问题的答案，用“前面子问题的结果”推导“后面子问题的结果”，避免重复计算（核心是找状态转移方程）。

适用场景：爬楼梯、打家劫舍、最长递增子序列、背包问题、股票问题。

```
// Demo：爬楼梯（经典DP，每次可走1或2步，求走到第n步的方法数）
int climbStairs(int n) {
    if (n == 1) return 1;
    int[] dp = new int[n+1]; // dp[i] = 走到第i步的方法数
    dp[1] = 1; // 初始状态：第1步1种方法
    dp[2] = 2; // 初始状态：第2步2种方法
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2]; // 状态转移：第i步 = 第i-1步 + 第i-2步
    }
    return dp[n];
}
```

10. 贪心

核心逻辑：每一步都做出“当前最优”的选择，不回头、不考虑后续影响，最终得到全局最优（需满足贪心选择性质）。

适用场景：分饼干、跳跃游戏、区间调度。

```java
// Demo：跳跃游戏（判断能否从第0位跳到最后一位）
boolean canJump(int[] nums) {
    int maxReach = 0; // 记录能跳到的最远距离
    for (int i = 0; i < nums.length; i++) {
        if (i > maxReach) return false; // 当前位置跳不到，直接返回false
        maxReach = Math.max(maxReach, i + nums[i]); // 更新最远距离
        if (maxReach >= nums.length - 1) return true; // 能跳到终点，提前返回
    }
    return true;
}
```

11. 单调栈

核心逻辑：维护一个“递增/递减”的栈，栈内元素有序，快速找到当前元素的“下一个更大/更小元素”，时间复杂度O(n)。

适用场景：柱状图最大面积、下一个更大元素、接雨水。

```java
// Demo：下一个更大元素（返回每个元素右边第一个比它大的元素，没有则为-1）
int[] nextGreaterElement(int[] nums) {
    int[] res = new int[nums.length];
    Stack<Integer> st = new Stack<>();
    // 从后往前遍历
    for (int i = nums.length-1; i &gt;= 0; i--) {
        // 栈内元素 <= 当前元素，弹出（不符合“更大”）
        while (!st.isEmpty() && st.peek() <= nums[i]) {
            st.pop();
        }
        res[i] = st.isEmpty() ? -1 : st.peek(); // 栈空则无更大元素
        st.push(nums[i]); // 当前元素入栈，供前面元素判断
    }
    return res;
}
```

12. 并查集（Union-Find）

核心逻辑：将元素分组，快速查询两个元素是否在同一组、合并两个组（核心是路径压缩和按秩合并，优化效率）。

适用场景：朋友圈、连通图、冗余连接。

```java
// Demo：并查集核心实现（路径压缩+按秩合并）
class UnionFind {
    int[] parent; // 记录每个元素的父节点
```

```java
    int[] rank; // 记录每个组的秩（高度）
    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i; // 初始：每个元素自己是父节点
            rank[i] = 1; // 初始：每个组的秩为1
        }
    }
    // 查找根节点（路径压缩）
    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // 递归压缩路径
        }
        return parent[x];
    }
    // 合并两个组（按秩合并）
    public void union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX == rootY) return; // 已在同一组
        // 秩小的组合并到秩大的组
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else {
            parent[rootY] = rootX;
            if (rank[rootX] == rank[rootY]) {
                rank[rootX]++;
            }
        }
    }
}
```

（注：文档部分内容可能由 AI 生成）